

Travelling Salesman Problem Using Greedy Technique and Simulated Annealing

Offline 4

Implementation Language: Python

1 Objective and Problem Representation

The objective is to find a minimum-cost tour that starts at City 0, visits every city exactly once, and returns to City 0. The input is an integer N followed by an $N \times N$ cost matrix, which may be symmetric or asymmetric. Internally, a tour is a Python list containing each city once. The closing return to City 0 is implicit and is added when the complete tour is costed or displayed.

Two methods were implemented: deterministic nearest-neighbor Greedy and probabilistic Simulated Annealing (SA). Greedy constructs a feasible tour quickly, whereas SA can accept some worse moves to escape local optima. Neither heuristic guarantees the global optimum.

2 Implementation

2.1 Greedy nearest neighbor

Starting from City 0, the algorithm repeatedly selects the cheapest unvisited city from the current city, appends it to the tour, and removes it from the unvisited set. After every city is visited, the return edge to City 0 is included in the cost. The locally cheapest choice can make later edges expensive, which motivates comparison with SA.

2.2 Simulated Annealing

SA supports either a random permutation of Cities 1 through $N - 1$ or the Greedy tour as its initial solution. A neighbor is generated using the required 2-opt operation: two positions $1 \leq i < j < N$ are selected and the segment between them is reversed. City 0 therefore stays fixed and no city is added or removed.

For a candidate tour, let $\Delta = C_{\text{new}} - C_{\text{current}}$. A move with $\Delta \leq 0$ is always accepted. A worse move is accepted with probability

$$P = \exp\left(-\frac{\Delta}{T}\right).$$

After the configured number of trials at a temperature level, geometric cooling applies $T \leftarrow \alpha T$. The search stops at the minimum temperature or maximum iteration limit, and a separate best-so-far tour is retained throughout.

Table 1: Main implementation functions

Function	Purpose
<code>readInput</code>	Reads N and the cost matrix from a file.
<code>calculateTourCost</code>	Adds all consecutive edges and the closing return edge.
<code>greedyTSP</code>	Constructs the nearest-neighbor tour from City 0.
<code>generateRandomTour</code>	Generates a random valid tour while keeping City 0 fixed.
<code>generateNeighbor</code>	Applies a 2-opt segment reversal.
<code>simulatedAnnealing</code>	Performs acceptance, cooling, stopping, and best-tour tracking.
<code>printResults</code>	Displays all required results and SA statistics.
<code>validateTour</code>	Checks the start, length, uniqueness, and completeness of a tour.

The program accepts one or more input filenames on the command line; with no filename it processes every `.txt` file in `inputs`. For both algorithms it displays the name, closed tour, total cost, and execution time. For SA it additionally displays the initial solution and cost, initial temperature, cooling rate, iterations, best tour and cost, accepted moves, and accepted worse moves.

3 Experimental Method

Five symmetric matrices were self-generated for $N \in \{10, 20, 50, 100, 200\}$. Off-diagonal integer costs were sampled from 1 to 1000 and mirrored across the diagonal; diagonal values were zero. The reproducible generator seed was $2026 + N$, so no external dataset citation is required.

Every stochastic configuration was run five times using seeds 1–5. Experiments 2 and 3 used Greedy initialization so that one parameter changed at a time; Experiment 4 compared both initialization methods. Values labelled “average” and average time summarize five runs. Timing was measured with `time.perf_counter` and is machine-dependent. When comparing different graph sizes, mean improvement is the mean of the five per-instance percentage improvements, avoiding domination by the larger raw costs.

4 Results and Discussion

4.1 Experiment 1: Greedy versus Simulated Annealing

The selected SA configuration ($T_0 = 100$, $\alpha = 0.995$, Random initialization) was run five times per instance. Percentage improvement uses the best SA cost:

$$\text{Improvement} = \frac{\text{Greedy Cost} - \text{SA Best Cost}}{\text{Greedy Cost}} \times 100.$$

Table 2: Greedy and SA comparison

Instance	Cities	Greedy	SA Best	SA Avg.	Improve.	G Time (s)	SA Time (s)
Test 1	10	1429	1311	1311.0	8.26%	0.000040	0.423053
Test 2	20	2430	1583	1614.0	34.86%	0.000064	0.461524
Test 3	50	4049	2371	2464.2	41.44%	0.000238	0.640037
Test 4	100	4289	3216	3543.8	25.02%	0.000568	0.927474
Test 5	200	6156	5493	5689.8	10.77%	0.002189	1.547525

SA produced a lower best cost than Greedy on every instance. Mean improvement was 24.07%, with the largest improvement, 41.44%, at 50 cities. Greedy remained much faster because it constructs one tour, while SA evaluates many neighbors. Thus Greedy favors speed, while SA gave better solution quality on these instances.

4.2 Experiment 2: Initial temperature

The tested values were $T_0 \in \{100, 1000, 5000\}$ with $\alpha = 0.995$ and Greedy initialization. Each city-size column below is the average final cost over five runs. “Mean worse” and time average the five instance-level averages.

Table 3: Effect of initial temperature on all five instances

T_0	$N = 10$	$N = 20$	$N = 50$	$N = 100$	$N = 200$	Mean improve.	Mean worse	Mean time (s)
100	1311.0	1645.0	2521.8	3651.0	5351.8	21.24%	295.44	0.790549
1000	1311.0	1601.0	2553.8	3795.6	6152.2	18.17%	9198.44	0.837538
5000	1311.0	1631.8	2905.8	4289.0	6156.0	13.87%	23230.36	0.877346

Higher temperature sharply increased acceptance of worse moves, from 295.44 on average at $T_0 = 100$ to 23,230.36 at $T_0 = 5000$, confirming greater exploration. However, excessive exploration reduced final quality within the fixed 100,000-iteration budget. $T_0 = 100$ had the highest mean improvement, gave the lowest average cost for 50, 100, and 200 cities, and tied at 10; $T_0 = 1000$ was better only at 20 cities. All settings reached the iteration limit, so time changed only slightly. Therefore $T_0 = 100$ was selected.

4.3 Experiment 3: Cooling rate

Cooling rates 0.90, 0.95, and 0.995 were tested with $T_0 = 100$ and Greedy initialization. Cost columns again show five-run averages.

Table 4: Effect of cooling rate on all five instances

α	$N = 10$	$N = 20$	$N = 50$	$N = 100$	$N = 200$	Mean improve.	Iterations	Mean time (s)
0.900	1358.2	1800.0	2879.6	3761.4	4876.4	18.57%	11000	0.089155
0.950	1311.0	1755.2	2784.4	3637.6	4930.0	20.47%	22500	0.180645
0.995	1311.0	1645.0	2521.8	3651.0	5351.8	21.24%	100000	0.790364

Fast cooling (0.90) stopped after 11,000 iterations and was cheapest computationally; slow cooling (0.995) used the full budget and took about 8.9 times longer on average. No rate dominated every size: 0.995 gave the best or tied average at 10, 20, and 50 cities, 0.95 was best at 100, and 0.90 was best at 200. Nevertheless, 0.995 achieved the highest mean percentage improvement (21.24%). Since solution quality was prioritized and even the 200-city runs remained practical, $\alpha = 0.995$ was selected.

4.4 Experiment 4: Random versus Greedy initialization

Both methods were run five times on every instance using $T_0 = 100$ and $\alpha = 0.995$.

Table 5: Effect of initialization method on all five instances

Test	Cities	Initialization	Initial Avg.	Best	Average	Worst	Avg. Time (s)
1	10	Random	4153.2	1311	1311.0	1311	0.389450
1	10	Greedy	1429.0	1311	1311.0	1311	0.385346
2	20	Random	10283.2	1583	1614.0	1662	0.454807
2	20	Greedy	2430.0	1583	1645.0	1692	0.455273
3	50	Random	23872.6	2371	2464.2	2597	0.647709
3	50	Greedy	4049.0	2427	2521.8	2618	0.649859
4	100	Random	50839.0	3216	3543.8	3794	0.980347
4	100	Greedy	4289.0	3390	3651.0	3830	0.941480
5	200	Random	98307.6	5493	5689.8	5950	1.623814
5	200	Greedy	6156.0	4826	5351.8	5875	1.566978

Greedy always supplied a much cheaper starting tour. After SA, Random initialization tied at 10 cities and had lower average costs at 20, 50, and 100 cities by 1.88%, 2.28%, and 2.94%, respectively; Greedy was 6.32% better at 200 cities. Best and worst results followed the same general pattern, and average times were close. Giving each size equal weight, Random had a small 0.16% mean normalized advantage and won more sizes, so it was selected. The 200-city result shows that this choice is empirical rather than universal.

5 Final Parameter Choices

Table 6: Selected Simulated Annealing configuration

Parameter	Value	Reason
Initial temperature T_0	100	Best mean improvement and strongest average quality on most sizes.
Cooling rate α	0.995	Best mean improvement, accepting the higher but practical run-time.
Initial solution	Random	Better average on three sizes, one tie, and a small normalized overall gain.
Minimum temperature	0.001	Low-temperature stopping threshold.
Iterations per level	100	Repeated sampling before each cooling step.
Maximum iterations	100000	Fixed upper bound on search effort.
Default random seed	42	Reproducible normal runs; experiments used seeds 1–5.

These settings are justified by the generated instances and are not claimed to be optimal for every TSP dataset.

6 Complexity Analysis

For Greedy, approximately N selections each scan up to N unvisited cities, giving $O(N^2)$ time and $O(N)$ extra space.

In SA, copying and reversing a segment is $O(N)$ in the worst case, and the implementation recalculates the whole closed-tour cost in $O(N)$. For I iterations, the search is therefore $O(I \times N)$. Random initialization is $O(N)$; Greedy initialization makes the total $O(N^2 + IN)$. Working tours use $O(N)$ extra space, while the stored input matrix makes total program space $O(N^2)$.

For a symmetric matrix, the cost difference of a 2-opt move could be computed from only its boundary edges, approaching $O(1)$ cost evaluation per proposal. The straightforward $O(N)$ recalculation was retained because it is easier to verify and also works correctly for asymmetric matrices, where reversing a segment changes internal edge directions.

7 Correctness and Limitations

City 0 remains fixed because random generation permutes only Cities 1 to $N - 1$ and 2-opt indices begin at 1. Segment reversal preserves every city exactly once. The cost function explicitly adds the last-city-to-City-0 edge, and `validateTour` checks length, starting city, uniqueness, and completeness. Development checks covered valid and invalid tours, symmetric and asymmetric costs, Greedy behavior, random tours, 2-opt neighbors, SA acceptance and stopping, and required output.

SA is stochastic, so results depend on the seed. Five repeated runs reduce this effect but do not eliminate it. The experiments use reproducible random symmetric matrices rather than standard benchmarks, and timing is hardware-dependent. Parameter interactions were limited by changing one factor at a time, so the selected configuration is a supported practical choice, not a proof of global parameter optimality.

8 Conclusion

The implementation satisfies the required input and tour representation, nearest-neighbor Greedy method, 2-opt neighborhood, SA acceptance rule, geometric cooling, configurable stopping parameters, required output, and correctness checks. Greedy was consistently faster, while the selected SA configuration found a better best tour on all five instances, averaging 24.07% improvement. Experiments across all five graph sizes justify $T_0 = 100$, $\alpha = 0.995$, and Random initialization while also identifying their tradeoffs.